

Relatório Técnico: Sistemas de Comunicação Indireta – Publish/Subscribe

1. Introdução aos Sistemas Publish/Subscribe

Os sistemas publicar-assinar, também denominados sistemas baseados em eventos distribuídos, operam sob uma lógica onde entidades editoras (publicadores ou *publishers*) disseminam eventos estruturados para um serviço de evento. Por outro lado, os assinantes (*subscribers*) manifestam seu interesse em receber eventos específicos por meio de assinaturas.

O principal papel do sistema é atuar como um intermediário que associa as assinaturas aos eventos publicados, assegurando a entrega correta das notificações. Como um único evento pode ser entregue a múltiplos assinantes interessados, este é fundamentalmente um modelo de comunicação do tipo um-para-muitos.

2. Principais Características e Vantagens

Heterogeneidade: A utilização de notificações de eventos como veículo de comunicação permite a integração de componentes distribuídos que não foram originalmente projetados para operar em conjunto. Basta que objetos geradores publiquem seus tipos de eventos e os interessados assinem os padrões correspondentes.

Assincronismo (Desacoplamento): As notificações fluem de maneira assíncrona entre publicadores e assinantes. Esse mecanismo garante que os produtores de eventos não precisem estar sincronizados (acoplados) com os consumidores de eventos.

3. Modelo de Programação

O modelo operacional dos sistemas Publish/Subscribe é construído sobre um conjunto enxuto de operações:

publish(e): Operação usada pelos publicadores para disseminar o evento *e* no sistema.

subscribe(f): Operação usada pelos assinantes para expressar interesse por meio de um filtro *f*.

unsubscribe(f): Revoga o interesse de uma assinatura prévia.

notify(e): Utilizada pelo sistema para entregar o evento aos assinantes.

advertise(f) e unadvertise(f): Operações complementares onde publicadores declaram antecipadamente a natureza dos eventos que pretendem gerar.

4. Modelos de Assinatura

A expressividade de um sistema Publish/Subscribe é diretamente definida pela forma como os filtros das assinaturas são construídos:

Modelo de Assinatura	Características
Baseado em Canal	Publicadores enviam para canais nomeados fixos e assinantes se inscrevem nestes canais físicos. É a estratégia mais primitiva.
Baseado em Tópico	As notificações possuem campos, sendo um deles o "tópico". A assinatura expressa interesse no tópico declarado explicitamente na mensagem.
Baseado em Conteúdo	Generaliza os tópicos permitindo criar filtros como consultas compostas por restrições sobre os valores dos atributos dos eventos, oferecendo maior expressividade.
Baseado em Tipo	Filtros baseados na tipagem dos eventos, altamente ligados a sistemas orientados a objetos. Permite a verificação de corretude de tipo diretamente na linguagem de programação.

5. Arquitetura e Implementação

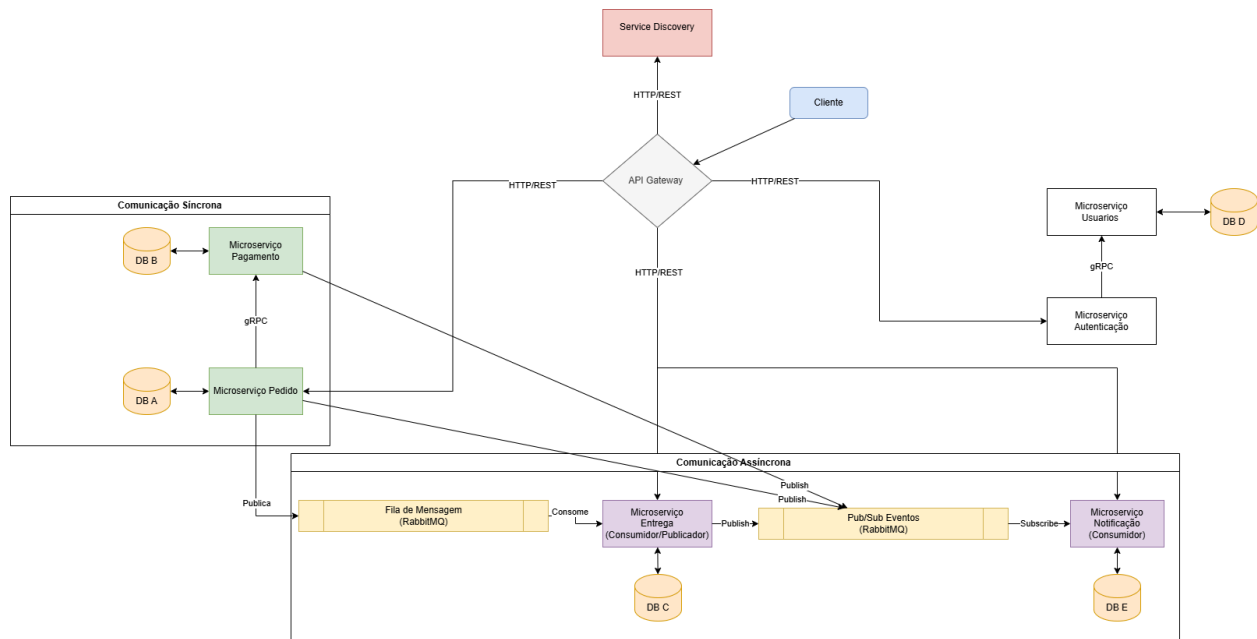
A implementação da tarefa de entregar os eventos de maneira eficiente pode ser classificada arquiteturalmente como:

Centralizada: Um servidor único no nó atua como intermediário de eventos (*broker*). Embora simples de implementar, cria um ponto único de falha e um gargalo de desempenho.

Distribuída (Rede de Intermediários ou Peer-to-Peer): O intermediário central é substituído por uma rede descentralizada, garantindo maior tolerância a falhas e escalabilidade. Todo nó pode cooperar no roteamento.

Para rotear os eventos nessa infraestrutura, utilizam-se técnicas como a **Inundação (Flooding)**, que envia a notificação para todos os nós da rede com foco na simplicidade, ou a **Filtragem**, onde os intermediários só encaminham mensagens por caminhos que levam a assinantes válidos. Uma ferramenta amplamente utilizada para prover essas funções na prática é o RabbitMQ, operando como o *broker*.

6. Estudo de Caso: Aplicação do Pub/Sub no Sistema de Delivery de Comida



O Papel do Pub/Sub na Arquitetura

Na camada inferior da arquitetura (Comunicação Assíncrona), é possível notar um componente centralizado intitulado **Pub/Sub Eventos (RabbitMQ)**. Esse *broker* atua como o intermediário principal para a disseminação de eventos de estado e de domínio entre os microserviços.

- **Os Publicadores (Publishers):** No diagrama, podemos observar que três serviços disparam a ação de *Publish* para o barramento de eventos: o **Microserviço Pedido**, o **Microserviço Pagamento** e o **Microserviço Entrega**. Cada vez que um estado crítico muda (por exemplo, "Pedido Criado", "Pagamento Aprovado" ou "Entrega Despachada"), esses microserviços enviam uma notificação de evento de forma totalmente independente e desacoplada.
- **O Assinante (Subscriber):** Do outro lado da arquitetura, encontra-se o **Microserviço Notificação**. Ele realiza o *Subscribe* no broker de eventos. Isso exemplifica perfeitamente a abordagem baseada em tópicos ou conteúdo abordada na teoria: o serviço de notificação assina o interesse em eventos relevantes, recebe a notificação (*notify*) assim que ela ocorre na malha distribuída, consulta sua própria base de dados e engatilha o aviso para o usuário.